Three.js project assets

For new or existing Three.js projects: preserve the project, starter identity, build configuration and
custom loaders. Do not reinitialize or run Godot doctor/import/bundling/export/release preparation.

Legacy build-size checks

Before building or saving an existing Three.js project, inspect `scripts/check-size.mjs` if present. Only edit an untouched official v1/v2 script with one of these SHA-256 digests:

- v1: `64edc9349af0a26881eb311aabdac518477c31786d0e15911a06b4440df5e502`
- v2: `f255f0da076b668f19fe2daa0a96b20455d736e10c77c7a61936ffe3a1c818ce`

In your normal editing task, remove only `transfer: 4.4 * MB` from `BUDGET` and `transfer` from `Object.entries({ js, fonts, transfer })`. Preserve JS/font budgets, total size reporting, source files and template identity. Tell the user about this change and include it in the next Manus checkpoint.
Preserve custom, missing or symlinked checks, including a symlinked parent directory. If editing fails,
report the blocker without claiming the limit was removed. Attachment and runtime recovery do not
modify this file. New v3 projects already omit the aggregate transfer rejection.

If a build-size check fails, report its measured size and overrun; preserve valid manifest entries and
respect user quality/scope constraints. Do not edit frozen starters or claim completion while builds fail.

Generation decision

New games follow the saved Blueprint model choices, which own
model-production methods and exact generation settings. An approved generative choice authorizes its recorded
method for the initial Blueprint implementation only; it is not ongoing consent for later requests.
Select individual models from gameplay needs and recorded restrictions. Historical `custom` choices
follow their recorded restrictions. Pending/cancelled choices and missing historical fields are not
generation consent. Preserve narrower supplied-only, no-AI, budget and style restrictions.

This decision does not authorize generated audio; assess need and obtain its separate approval.

Existing games may add models without replacing their Blueprint or restarting setup. A direct request to
generate a model authorizes that model; a generic request to improve visuals does not. Settle only the
new asset decision using the existing Blueprint when it is still pending, or one focused question when
there is no pending card. Preserve settled gameplay and online services. Do not recreate the game.

Generate, validate, register

Resolve `GAME_RUNTIME` only from the current init/attach/run-start receipt. Before paid generation, read
`$GAME_RUNTIME/asset-capabilities.json`. Require `engines` to include `threejs` and `manifestVersion: 2`.
Read this file before running any script: an older script may ignore unknown flags and perform work.
After that check, `node "$GAME_RUNTIME/sync-assets.mjs" --capabilities` is an optional read-only probe.
If the installed package cannot report this, stop before generation and update the
Addon runtime, then retry in the same project. New UI or a refreshed tool list alone is not proof that
the device package has activated. Preserve the current task and files during update.

For authorized generation, follow shared 3D generation for the
built-in default, explicit connector selection and retry brief preservation. An enabled connector or a mention
of Tripo does not select that route; do not add a connector-choice question or timed default. Preserve an explicit
entry-point choice already made for this request. Before generation, state whether the selected tool deducts
Manus credits or credits from the user's connected provider account; connector use does not make the surrounding
Manus task free.

For the built-in choice, use `manus-tools.generate_3d_model` with an absolute `.glb` output path under
`assets/models/` in the bound project. Text-only uses `prompt`; single-image uses `image` without `prompt`.

For an intentional reference edit, use an image tool first, then pass the completed image; preserve originals.
Follow the tool's actual schema and the saved Blueprint recipe, including its handling of unsupported settings.
If it is unavailable, explain that the runtime/tool capability must update; do not retry initialization,
invoke an unregistered tool or silently switch to the connector.

Built-in game asset model preferences

Before each built-in `generate_3d_model` call, assign the model a detail tier within the approved text/image method:

- **Light**: scenery, foliage, terrain props, small or repeated pickups, distant models, and models that the brief
  or request calls low-poly, low-res or simple. Use P1 and omit `face_limit`.
- **Standard**: player characters, main enemies, bosses, NPCs, vehicles and other prominent close-camera models.
  Use P2 and omit `face_limit`.
- **High**: only models that the user explicitly wants highly detailed, high-poly or high-topology, or with
  detailed or extreme textures. Use P2 with the requested face count, otherwise the model's maximum, so High is
  never below Standard's adaptive count; set the requested `texture_quality`. Take each model's maximum
  from the tool's per-model `face_limit` description, not the JSON `maximum`; if a requested count exceeds it,
  use that maximum and tell the user.

A descriptor applies only to the models it describes: "a knight in a low-poly forest" makes the trees and rocks
Light and keeps the knight Standard. Quality uses these tiers. Fast and saved `hybrid` use only P1: their models
are Light, and High models keep P1 with P1's maximum `face_limit` and the requested `texture_quality`.
Later additions follow the saved choice without gaining generation consent. Legacy `ai_generated` and additions
without a saved Quality, Fast or Hybrid choice use Light unless the request prioritizes quality. A replacement
keeps the replaced model's tier unless the request changes its detail. It passes the same non-default arguments

again: the tier's `model`, any `face_limit` and requested `texture_quality`, and the saved mode's texture, PBR
and texture-version settings. Omitted arguments fall back to tool defaults (P1, PBR, standard textures).
Explicit user model, face-count, texture, budget and source instructions take precedence. Resolve exact model IDs and supported options from the active tool schema; account aliases are not proof of a
selectable model version. If the preferred model cannot be selected, report the limitation and defer those assets
rather than inventing arguments, switching to a connector or submitting paid capability probes.

Only after a confirmed P-model concurrency-limit rejection, consider H3.1 on the same built-in tool with
`smart_low_poly: true`, where the active schema supports it and recorded constraints permit. Preserve the brief,
reference and unrelated options; adapt only model-dependent incompatible settings and disclose the switch.
This exception permits switching the rejected P-model submission to H3.1, with its required low-poly option.
Do not switch for generic errors, timeouts or uncertain submission results. Recover accepted/in-flight/completed
jobs before another submission; never regenerate an existing result merely to change model. If H3.1 is
unavailable or conflicts with an explicit requirement, wait for capacity or report the deferred asset.

Explicit connector generation

For an explicit connector choice, follow manus-config's execution-context rules and discover the registered
generation/status/export tools through its supported tool-discovery path. If setup is needed, inspect the connector
with `manus-config connector search Tripo`, use the returned UID with `manus-config connector get <uid>` and
`manus-config connector ensure "Tripo" --uid <uid>`, and wait for any pending configuration card confirmation.
Older sandboxes follow manus-config's documented fallback while preserving pending edits. Never treat expiry as
permission to enable or reauthorize an account. Use the registered connector tools and their actual schemas to

generate, check status and export the selected model. Keep the task/result identity and download the
completed GLB into a new absolute path under the bound project's `assets/models/`. If export is needed,
export that existing result as GLB; do not regenerate or rename another format to `.glb`. Preserve any
referenced textures/buffers alongside the model. A connector connection/authorization failure is a
blocker to report, not permission to switch to Manus billing or create a new paid generation task.

For either entry point, generation and device delivery are separate. If generation succeeded but
delivery failed, recover the existing result from its artifact receipt or connector task/result identity;
do not generate or charge again. A session or provider URL is a transfer source only. Land the complete
model in the project and validate it before using it. No placeholder GLB
or in-progress manifest entry. Choose a new filename for each regenerated model,
preserve the old file and change gameplay references only when asked to apply the result.
Asset-panel requests create library assets; the user can use Send to Manus to apply them. Explicit
generate-and-apply requests can do both.

For early registration after files land, run `node "$GAME_RUNTIME/sync-assets.mjs" --engine=threejs`
from the project root; skip it when saving next unless the batch needs grouping first. In Local First,
run sync/save through `webdev.manus_git` to use the Managed Git index; in Cloud use the project terminal.
Sync validates files, uploads changes, restores missing recorded bytes and updates `assets.lock.json` v2,
reusing hashes/storage and preserving groups. Do not invent storage keys. On upload failure, retry with
the existing output; completed uploads survive. Apply Asset browser groups
to the registered batch before saving.

Save and load assets

Use `node "$GAME_RUNTIME/save-threejs.mjs" --message "Update"` with temporary Manus Git author/committer
identity before each checkpoint. Follow Git/checkpoints for remote integration,
push, receipt verification and uncertain-command recovery. Save returns a local SHA and

never pushes. If unavailable, update
the runtime in this project; do not replace it with a raw commit containing assets.

Save synchronizes assets, then commits source, lock, package.json, recovery scripts and .gitignore together.
Resolve staged edits or unfinished merge/rebase first. Sync installs `scripts/manus-game-assets/` before
existing build/dev commands and exact ignore rules for validated registered files. Early sync leaves the
index untouched; save removes tracked assets only from its private candidate index, preserving working
copies and Git history. Never force-add managed binaries or commit after failed synchronization.
Unsupported/unregistered inputs, including separate glTF buffers, remain in Git. No starter upgrade or
automatic compression is needed.

Use package build/dev commands, not direct Vite, to restore missing assets and verify download size/hash
before the original command. Existing local bytes are unchanged; sync/save treats them as edits. History
previews trust files in the selected commit and validate missing Storage downloads through the authorized
operation reader before Vite, without exposing credentials to project code. Hosted builds need
`MANUS_API_URL` / `MANUS_API_KEY`; downloads stream to disk with at most eight transfers and three transient
attempts, using Godot's preview retry/timeout policy. Permission/integrity failures stop restoration.
There is no additional asset baseline, Git-tracking fallback or asset revision counter.

Asset references

Existing local/bundler paths remain supported for restoration and editing; use stable Storage addresses for
player runtime media as described in staged CDN loading. Source, styles and JSON tables select restored assets. For
`import.meta.glob`, template paths like `assets/models/${name}.glb`, or concatenation after a literal
`assets/models/` prefix, restoration includes every registered file under that prefix, including nested
textures; glob exclusions do not narrow it. Unrelated library directories stay in Storage. Fully

computed
paths without a literal directory prefix must be listed explicitly in asset tables.
For stable `/manus-storage/...` addresses and temporary download URLs, follow
Storage; never persist signed URLs or credentials.
Saved previews freeze only storage paths referenced in the browser build, so retain
complete paths in
built code/tables, including dynamically selected models, rather than separate hash/path
fragments.

Staged CDN loading

Keep runtime media (models, textures, audio, UI art and fonts) in project Storage/CDN and
reference its stable
`/manus-storage/game/sha256/...` addresses from explicit asset tables. Use only paths that
sync/save registers
in `assets.lock.json`; saved-version previews admit no other Storage path. Land Storage
upload or
generated-image media in the project and register it through sync/save before referencing
it, or keep it in the
build output. Never persist signed URLs or upload registered bytes again merely for loading.
Keep editable source, locks and recovery intact. CDN registration removes managed
binaries from Git;
local imports and files copied from `public/` can still enlarge the deployed bundle. Exclude
CDN-backed copies
from build output and avoid base64/inlined media; verify the output rather than assuming
upload shrinks it.
This supersedes the starter's local `public/assets/` recommendation for runtime media;
preserve custom loaders
while adapting their URLs and dependencies.

Partition asset tables by title screen and wave, level, stage or scene. Cold boot waits only
for title/controls
and the first playable segment's complete dependencies, then opens the title immediately;
later content must
not join that boot barrier. Reuse the cached loader and prioritize current plus next segment.
Once those are
ready, prefetch subsequent segments with bounded concurrency and bounded decode,
GPU upload and instantiation
work per frame. Include textures, animations, audio and shader warmup needed for each
segment, not just models.
Keep external model buffers/textures reachable through their registered URLs and load

required decoders too.
Gate entry on that segment's readiness, with honest progress and recoverable Retry on required-asset failure.

Pause optional prefetch dispatch, decoding, uploads and warmup during dense combat/high VFX or actor counts,
sustained frame-time pressure, and hidden/suspended play; resume when pressure clears. In-flight transfers may
finish, but must not trigger heavy integration while paused. Required current/next loads retain priority.
Deduplicate requests, ignore completions for exited scenes and release unused resources within a memory budget.
Verify cold-cache title/first-play time, initial bytes and bundle size, later-segment transitions, failed loads,
frame pacing under crowded combat while prefetch pauses/resumes, and a saved-version preview. Claim measured
improvements only.

Version changes and delivery

Before switching Git versions, preserve unsaved asset edits. After materializing another checkpoint or
fast-forwarding to a saved tree, run `node "$GAME_RUNTIME/restore-threejs.mjs" --from-lock` before building,
editing or syncing. This verifies size/hash and restores the selected lock's bytes, including stale
same-name local copies, without upload or manifest changes. It is not a normal build step. During native forward rollback, preserve the exact target tree; do not sync or use standard save.

Before saving, require a passing build from a clean checkout of the candidate source and pinned lock
without ignored working copies, plus real Preview. For favicon/OG preparation, follow Game sharing; put their PNGs under `assets/` for registration and recovery.
ZIP/remix/copy are post-save operations, not checkpoint prerequisites. When requested, verify exact-version
restoration and destination storage transfer, reporting unverified steps. ZIP contains editable bytes but
does not provision destination storage; complete or verify that transfer before reusing its storage paths.
Never substitute another version or regenerate missing assets.

Asset browser groups

Choose groups by the game's browsing purpose, not media type, source pack, production method or directory depth.
`assetType` describes the media; `groupId` describes its purpose in the game. Type directories such as
`models`, `images` or `audio` are not useful purpose groups. Keep files in their existing paths; classify
subjects by their actual role rather than imposing a fixed taxonomy or a group for every folder.
Preserve existing user-defined IDs, names, memberships and order unless regrouping is requested;
reserved `assets/template/` media stays in `template`. Automatically inferred directory groups are defaults,
not user-defined categories: assign meaningful groups to the new batch even when sync already supplied one.

After the engine-specific sync has registered the new batch, edit only complete `assets.lock.json` entries.
Never prewrite partial entries with just `groupId` or invent storage facts. Already registered assets need
no extra sync for grouping alone. Assign `groupId` and matching `groups` declarations with human-readable
`name` and `order`. Normalize IDs to lowercase, replace runs of non-letter/non-number characters with `-`,
and trim edge hyphens. Keep recolored/adapted variants with their subject.

Runtime fallback uses the first directory below `assets/`, so `assets/models/` becomes `models` regardless
of content. This is registration fallback, not completed grouping. Change only membership and group
metadata, never storage paths, hashes, sizes, asset types or provenance; no file move or upload is needed.
Check the batch's entries and declarations before saving, then confirm the saved version retains the groups.
A post-save grouping edit needs a new checkpoint. No browser session is required.

Use and verify

Use the existing loader (or `GLTFLoader`); verify scale, pivot, orientation, textures, color space and
animations in Preview. Reuse geometry/materials, dispose retired resources, use separate

simple collision
geometry and show a placeholder on load/failure.
Instance rigged GLBs with `SkeletonUtils.clone()` (`three/examples/jsm/utils/SkeletonUtils.js`), not `.clone()` ,
which keeps the source skeleton so copies freeze or render on the original, and give each its own `AnimationMixer` .
Make each moving character, including enemies and NPCs, face its travel direction, unless the design aims it
elsewhere (for example at a lock-on target). Keep the last heading when it stops. glTF models usually face +Z,
but the Three.js starter's `player.ts` turns local -Z toward travel. Correct the front with one fixed yaw on a
wrapper group above the animated nodes. Then check the front in a moving Preview screenshot.

Two-handed weapon grips

Follow character/equipment separation and grip rigging.
Use supported built-in tools and suitable existing joints/attachments first. Only when these cannot meet an approved
grip requirement, read local rig augmentation fallback for missing joints/weights,
weapon sockets and secondary-hand IK. It is optional post-processing, never a default re-rig or paid retry.

3D rendering safety

Use restrained environmental reflections and emission: keep ground, walls, scenery and imported/generated props
mostly rough and non-emissive, with a deliberate roughness floor; reserve shine/glow for small intentional accents
and gameplay cues. Reduce environment/specular intensity, emissive strength and bloom together when glare appears.
Keep point lights away from glossy surfaces and bound light intensity, exposure and emissive output within a scene
brightness budget. Inspect the combined scene at representative camera angles; do not compensate with more bloom.

For an HDR/post-processing chain, put a per-pixel sanitize pass immediately after scene rendering and before bloom,
blur, depth of field, temporal accumulation or any other neighbouring-pixel read. Replace NaN/non-finite channels
with zero, then clamp finite HDR colour to a deliberate bright ceiling before output/tone

mapping. Read each source
texel without filtering contaminated neighbours. Use finite checks supported by the project's shader dialect;
`clamp` alone does not sanitize NaNs. Preserve required alpha/depth semantics and apply the ordering to every
affected scene/composer path, including quality changes. Add no post-processing solely to a direct-render path.

Keep custom shaders finite at their source: guard divisors and zero-length normalization; constrain `pow` bases
and exponents to a defined domain, `sqrt` to nonnegative inputs and `log` to positive inputs. Clamp `acos` inputs
to [-1, 1], guard the undefined zero-vector case of two-argument `atan`, keep `smoothstep` edges strictly ascending
(use `1 - smoothstep(low, high, x)` for a descending ramp), and clamp interpolated masks/weights to [0] [1].
Check intermediate overflow as well as final output. Sanitization contains bad pixels; fix the originating shader.

Camera-dependent black rectangles can indicate non-finite HDR pixels spreading through post effects; confirm
the source rather than assuming every rectangle has that cause. Verify zero/domain boundaries, bright highlights,
camera motion and quality changes with effects enabled/disabled; inspect scene and post-process buffers for
non-finite values and spreading artifacts. Report runtime checks as unverified when only guidance changed.

The built-in delivery path accepts up to 64 MiB; connector generation/export limits come from its tool
contract. For either entry point, the asset viewer uses the same 20 MiB per-file read limit for all asset
types. Model preview supports only its current GLB formats/extensions. A valid larger model may be
available as an asset without an inline preview.
Check required GLB extensions against the game's configured decoders; do not claim the asset viewer
supports every runtime format.